Robust Bank Account Management System: A C-Language Implementation

Foundations of Programming: Mini-Project

Instructor: Nikhil Soniminde

Team Members:

1. **Abhanshi Dubey**
   PRN: 1262253325
2. **Adit Kothari**
   PRN: 1262251552
3. **Sharvari Harbude**
   PRN: 1262250522
4. **Soham Kopurwar**
   PRN: 1262250813
5. **Ishan Khatti**
   PRN: 1262251872

Introduction: Project Overview

Core Banking Functionality Simulation

Developed to simulate essential banking operations, this module serves as a practical demonstration of fundamental C programming constructs, including data structures, file I/O for persistent storage, and control flow mechanisms.

Real-World Application & Relevance

This project showcases the direct applicability of C programming skills in constructing functional systems, illustrating how academic principles can effectively simulate practical, everyday financial management scenarios.

Structured Interactive Menu System

An intuitive, menu-driven command-line interface provides seamless user interaction and navigation through various banking functions, emphasizing modular design principles and robust input validation.

Project Overview: Problem & Objectives

Problem Statements

Manual tracking of transactions can be confusing and error-prone, leading to inefficiencies and potential inaccuracies.

Objectives

- Create a simple banking simulation system.
- Implement core C programming concepts, such as data structures and control flow.
- Develop logical and structured programming skills through practical application.

Features of the Program

Display Account Balance

Allows users to view their current account balance, reflecting all deposits and withdrawals. This feature demonstrates dynamic state management of a numerical value.

Process Fund Deposits

Enables users to add funds to their account. Input validation ensures that only positive amounts are accepted, and the account balance is updated accordingly.

Track Cumulative Deposits

Provides a historical record of the total sum of all funds ever deposited into the account, distinct from the current balance. This utilizes an accumulator variable for persistent tracking.

Manage Funds Withdrawal

Facilitates the removal of funds from the account. Critical checks are implemented to prevent overdrafts, ensuring the withdrawal amount does not exceed the available balance.

Monitor Cumulative Withdrawals

Displays the aggregate amount of all funds withdrawn from the account over time. Similar to deposits, this provides an overview of outgoing transactions.

Initiate Program Termination

Offers a controlled exit mechanism from the simulation. This ensures a clean termination of the application loop and signifies the end of the user session.

Core Concepts in C Programming

Data Types and Variables

Exploration of fundamental data types like `int` and `float`, and their role in declaring variables for efficient data storage and manipulation within C programs.

Conditional Control Flow (if-else)

Implementation of `if-else` constructs to execute specific blocks of code based on evaluated conditions, enabling essential decision-making logic in program execution.

Multi-way Branching with Switch-Case

Utilizing the `switch-case` statement for streamlined selection among multiple possible execution paths, optimizing complex conditional logic for clearer code.

Iterative Constructs: The do-while Loop

Application of the `do-while` loop, ensuring at least one iteration of a code block before evaluating the loop condition, suitable for specific repetitive tasks.

Fundamental Arithmetic Operations

Execution of basic mathematical computations, including addition, subtraction, multiplication, and division, leveraging C's built-in operators for numerical processing.

Standard Input with scanf()

Facilitating interactive program execution by securely acquiring and parsing user-provided data from standard input using the versatile `scanf()` function.

Program Execution Flow

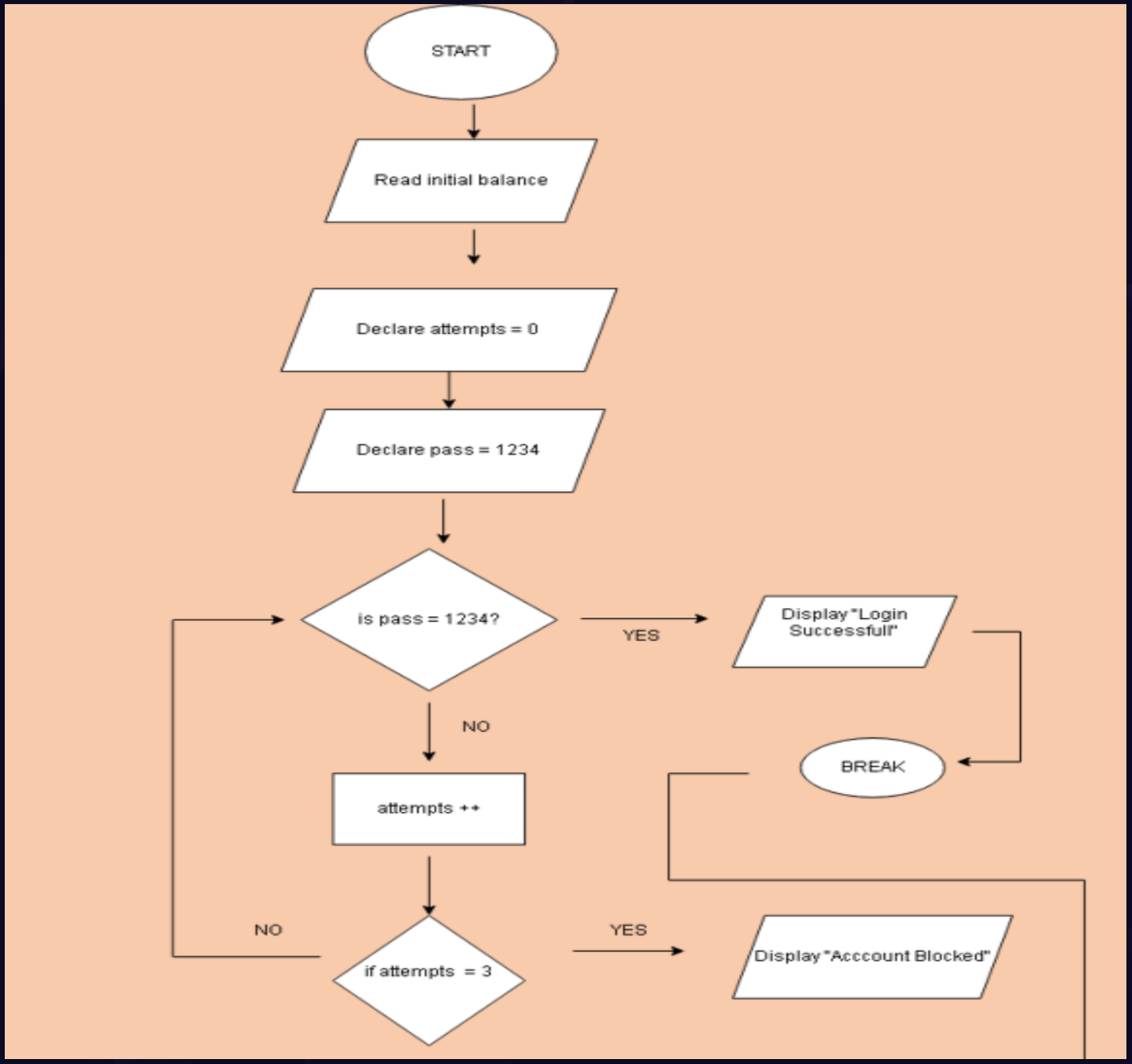
The program initializes, prompting the user to input their initial account balance.

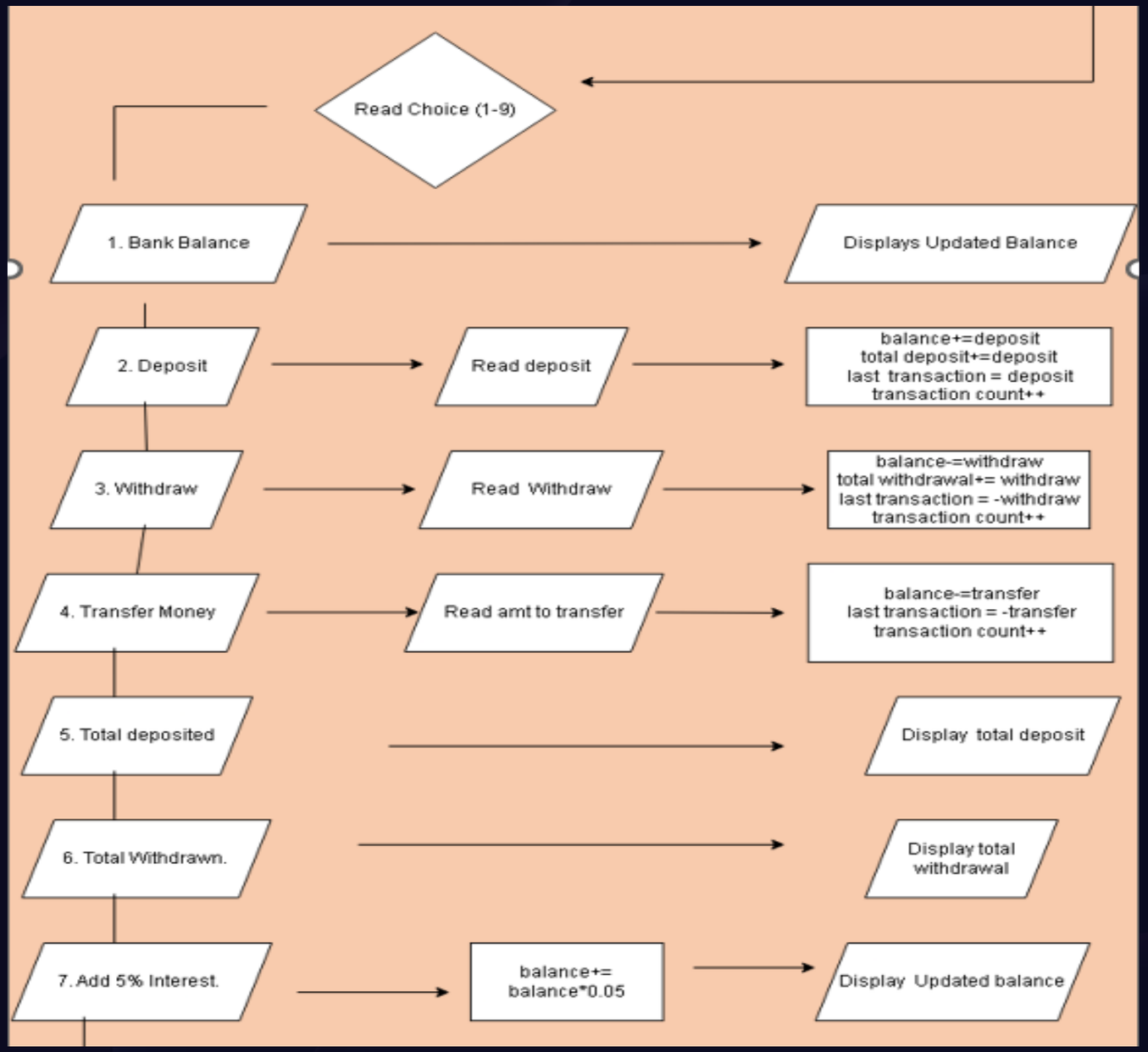
A main menu is presented to the user, offering various transaction options.

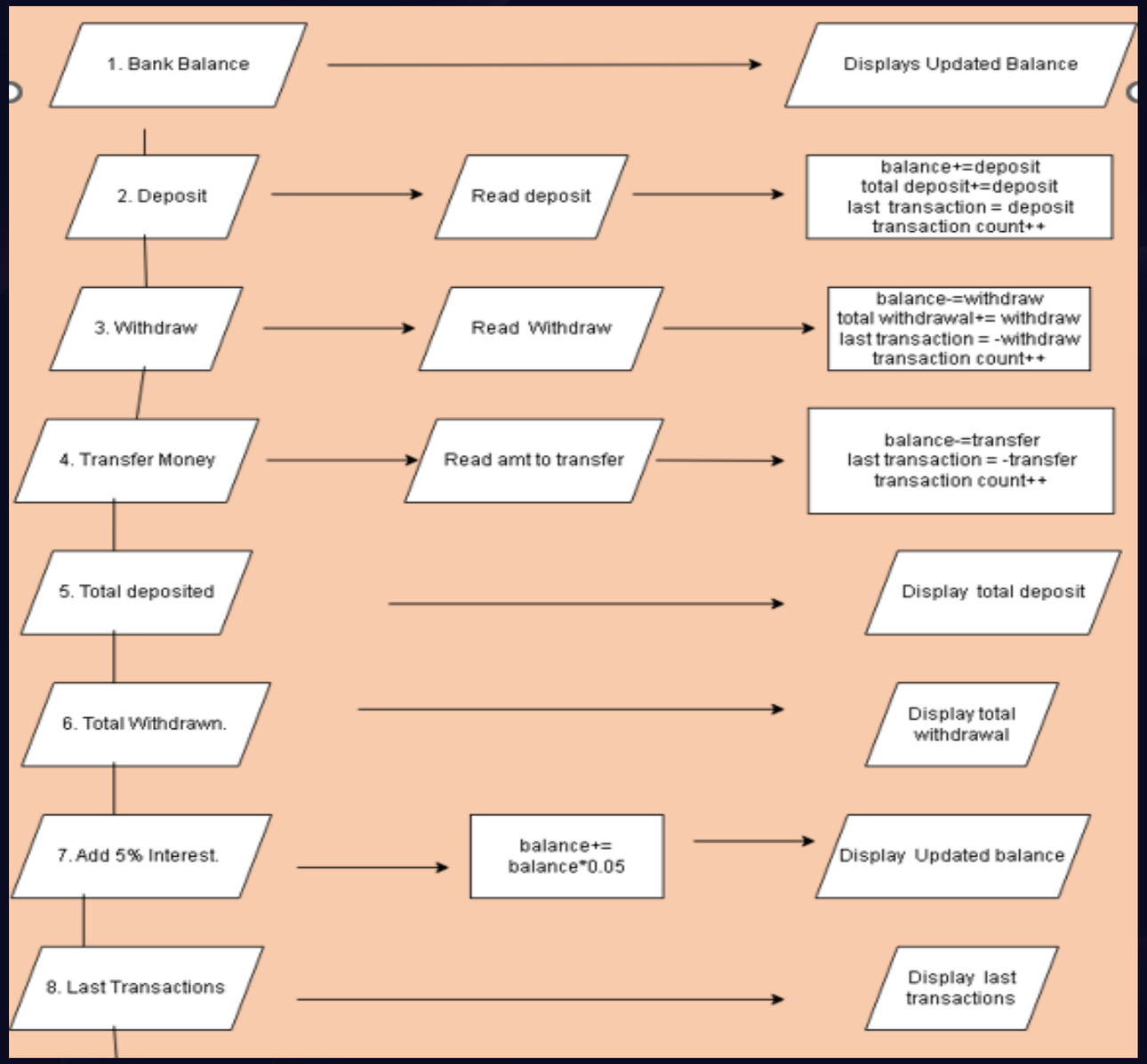
The user selects a desired operation from the presented menu.

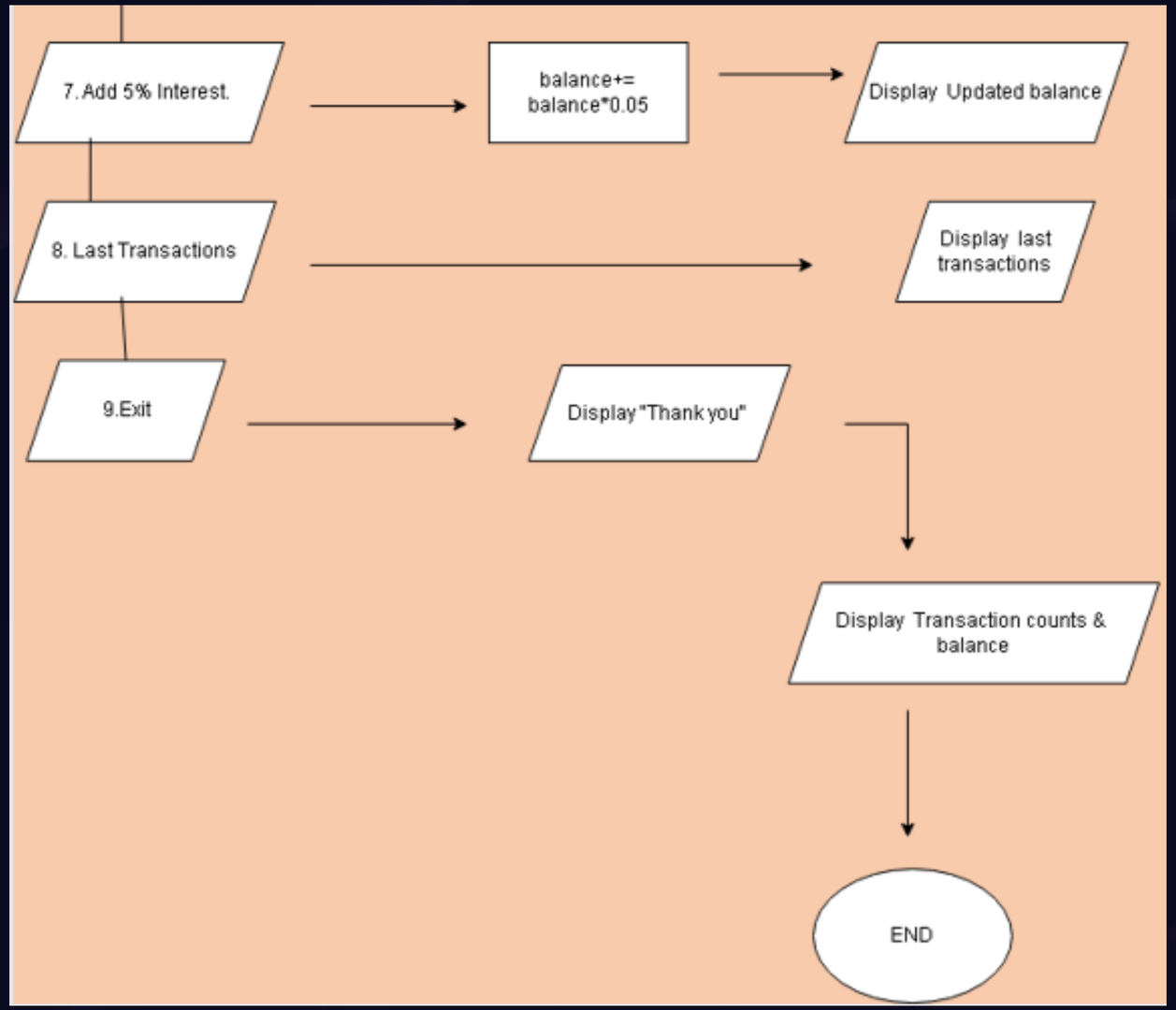
The corresponding operation (e.g., deposit, withdrawal, balance inquiry) is executed, updating the account state as required.

The execution loop persists, allowing continuous transactions, until the user explicitly chooses the 'Exit' option.


START
Read initial balance
Declare attempts = 0
Declare pass = 1234
is pass = 1234?
YES
Display "Login Successfull"
NO
BREAK
attempts ++
NO
if attempts = 3
YES
Display "Acccount Blocked"


Read Choice (1-9)
1. Bank Balance
Displays Updated Balance
2. Deposit
Read deposit
balance+=deposit
total deposit+=deposit
last transaction = deposit
transaction count++
3. Withdraw
Read Withdraw
balance-=withdraw
total withdrawal+= withdraw
last transaction = -withdraw
transaction count++
4. Transfer Money
Read amt to transfer
balance-=transfer
last transaction = -transfer
transaction count++
5. Total deposited
Display total deposit
6. Total Withdrawn.
Display total withdrawal
7. Add 5% Interest.
balance+= balance*0.05
Display Updated balance


1. Bank Balance
Displays Updated Balance
2. Deposit
Read deposit
balance+=deposit
total deposit+=deposit
last transaction = deposit
transaction count++
3. Withdraw
Read Withdraw
balance-=withdraw
total withdrawal+= withdraw
last transaction = -withdraw
transaction count++
4. Transfer Money
Read amt to transfer
balance-=transfer
last transaction = -transfer
transaction count++
5. Total deposited
Display total deposit
6. Total Withdrawn.
Display total withdrawal
7. Add 5% Interest.
balance+= balance*0.05
Display Updated balance
8. Last Transactions
Display last transactions


7. Add 5% Interest.
balance+= balance*0.05
Display Updated balance
8. Last Transactions
Display last transactions
9.Exit
Display "Thank you"
Display Transaction counts & balance
END

Bank Account Simulation Code

```c
#include <stdio.h>

int main()
{

    printf("\n*********  MINI ATM SYSTEM  *********\n");

    float balance;      // Variable to store account balance
    int choice;         // Variable to store user menu choice

    // Take initial balance from user
    printf("\nEnter Initial Balance: ");
    scanf("%f", &balance);

    /* ---------------- PIN SYSTEM ---------------- */
    int pin = 1234;          // Default PIN
    int enteredPin;          // Variable to store user entered PIN
    int attempts = 0;        // Counter for number of attempts

    // Allow maximum 3 attempts
    while (attempts < 3)
    {
        printf("Enter 4-digit PIN: ");
        scanf("%d", &enteredPin);

        if (enteredPin == pin)
        {
            printf("Login Successful!\n");
            break;          // Exit loop if PIN is correct
        }
        else
        {
            printf("Wrong PIN!\n");
            attempts++;     // Increase attempt counter
        }
    }
```

```c
// If user enters wrong PIN 3 times
if (attempts == 3)
{
    printf("Too many failed attempts. Account Locked!\n");
    return 0;           // Terminate program
}

/* -------- Transaction Tracking Variables -------- */
float totalDeposit = 0;        // Stores total deposited amount
float totalWithdrawal = 0;     // Stores total withdrawn amount
float deposit, withdraw, transfer; // Temporary variables
int transactionCount = 0;      // Counts number of transactions
float lastTransaction = 0;     // Stores last transaction amount

/* ------------- MENU DRIVEN LOOP ------------- */
do
{
    printf("\n----------------------------------\n");
    printf("1. Check Balance\n");
    printf("2. Deposit\n");
    printf("3. Withdraw\n");
    printf("4. Transfer Money\n");
    printf("5. Total Deposited\n");
    printf("6. Total Withdrawn\n");
    printf("7. Add 5%% Interest\n");
    printf("8. Last Transaction\n");
    printf("9. Exit\n");
    printf("---------------------------------\n");

    printf("Enter your choice: ");
    scanf("%d", &choice);
```

```c
switch (choice)
{
case 1:
    // Display current balance
    printf("Current Balance: %.2f\n", balance);
    break;

case 2:
    // Deposit Money
    printf("Enter amount to deposit: ");
    scanf("%f", &deposit);

    if (deposit > 0)   // Validate amount
    {
        balance += deposit;             // Add to balance
        totalDeposit += deposit;        // Update total deposit
        lastTransaction = deposit;      // Store last transaction
        transactionCount++;             // Increase transaction count
        printf("Deposit Successful!\n");
    }
    else
        printf("Invalid amount!\n");
    break;

case 3:
    // Withdraw Money
    printf("Enter amount to withdraw: ");
    scanf("%f", &withdraw);

    if (withdraw > 0 && withdraw <= balance)  // Check valid amount
    {
```

```c
case 5:
    // Show total deposited amount
    printf("Total Deposited: %.2f\n", totalDeposit);
    break;

case 6:
    // Show total withdrawn amount
    printf("Total Withdrawn: %.2f\n", totalWithdrawal);
    break;

case 7:
    // Add 5% interest to balance
    balance += balance * 0.05;
    printf("5%% Interest Added!\n");
    printf("Updated Balance: %.2f\n", balance);
    break;

case 8:
    // Show last transaction
    if (transactionCount == 0)
        printf("No transactions yet.\n");
    else
        printf("Last Transaction Amount: %.2f\n", lastTransaction);
    break;

case 9:
    // Exit option
    printf("\nThank You for Banking With Us!\n");
    break;
```

```c
        case 9:
            // Exit option
            printf("\nThank You for Banking With Us!\n");
            break;

        default:
            // Handle invalid input
            printf("Invalid Choice! Select 1-9.\n");
        }

    } while (choice != 9);  // Loop until user chooses Exit

    // Display final summary
    printf("\nTotal Transactions Done: %d\n", transactionCount);
    printf("Final Balance: %.2f\n", balance);

    return 0;
```

Output

Quantitative Analysis Results

```
Enter Initial Balance: 5000
Enter 4-digit PIN: 1234
Login Successful!

---------------------------------
1. Check Balance
2. Deposit
3. Withdraw
4. Transfer Money
5. Total Deposited
6. Total Withdrawn
7. Add 5% Interest
8. Last Transaction
9. Exit
---------------------------------
Enter your choice: 1
Current Balance: 5000.00
```

System Architecture Blueprint

```
---------------------------------
1. Check Balance
2. Deposit
3. Withdraw
4. Transfer Money
5. Total Deposited
6. Total Withdrawn
7. Add 5% Interest
8. Last Transaction
9. Exit
---------------------------------
Enter your choice: 2
Enter amount to deposit: 6767
Deposit Successful!
```

Interactive Prototype Snapshot

```
---------------------------------
1. Check Balance
2. Deposit
3. Withdraw
4. Transfer Money
5. Total Deposited
6. Total Withdrawn
7. Add 5% Interest
8. Last Transaction
9. Exit
---------------------------------
Enter your choice: 3
Enter amount to withdraw: 6767
Withdrawal Successful!
```

Key Visual Outputs and Interpretations

```
1. Check Balance
2. Deposit
3. Withdraw
4. Transfer Money
5. Total Deposited
6. Total Withdrawn
7. Add 5% Interest
8. Last Transaction
9. Exit
---------------------------------
Enter your choice: 4
Enter amount to transfer: 67
Transfer Successful!
```

```
1. Check Balance
2. Deposit
3. Withdraw
4. Transfer Money
5. Total Deposited
6. Total Withdrawn
7. Add 5% Interest
8. Last Transaction
9. Exit
---------------------------------
Enter your choice: 5
Total Deposited: 6767.00
```

```
1. Check Balance
2. Deposit
3. Withdraw
4. Transfer Money
5. Total Deposited
6. Total Withdrawn
7. Add 5% Interest
8. Last Transaction
9. Exit
---------------------------------
Enter your choice: 6
Total Withdrawn: 6767.00
```

```
1. Check Balance
2. Deposit
3. Withdraw
4. Transfer Money
5. Total Deposited
6. Total Withdrawn
7. Add 5% Interest
8. Last Transaction
9. Exit
---------------------------------
Enter your choice: 7
5% Interest Added!
Updated Balance: 5179.65
```

```
1. Check Balance
2. Deposit
3. Withdraw
4. Transfer Money
5. Total Deposited
6. Total Withdrawn
7. Add 5% Interest
8. Last Transaction
9. Exit
---------------------------------
Enter your choice: 8
Last Transaction Amount: -67.00
```

```
1. Check Balance
2. Deposit
3. Withdraw
4. Transfer Money
5. Total Deposited
6. Total Withdrawn
7. Add 5% Interest
8. Last Transaction
9. Exit
---------------------------------
Enter your choice: 9

Thank You for Banking With Us!

Total Transactions Done: 3
Final Balance: 5179.65
```

Core Logic: Transaction Management and Program Control

Deposit Mechanism

Funds are added directly to the account balance using the operation: `balance += deposit`. This ensures an immediate and atomic increment upon successful transaction completion.

Withdrawal Mechanism

Funds are precisely deducted from the account balance via: `balance -= withdraw`. This operation is critically executed only after a stringent balance validation check.

Pre-withdrawal Balance Validation

A fundamental conditional check, expressed as `if(balance < withdraw)`, is implemented to strictly prevent any transaction that would result in an insufficient account balance, thereby safeguarding financial integrity.

Interactive Session Management

The application's execution is sustained within a continuous program loop, which persists interactively until the user explicitly initiates termination by selecting option 6 (Exit).

Conclusion

Banking System Simulation

Successfully implemented a console-based banking system, providing a robust simulation of core financial transactions including deposits, withdrawals, and balance inquiries.

Fundamental C Programming Mastery

Demonstrated proficient application of essential C programming concepts such as control structures, data types, and function calls to construct reliable system logic.

Structured Programming Principles

Utilized structured programming methodologies, ensuring a modular design and clear separation of concerns, which enhances code readability, maintainability, and future scalability.

Envisioning Future Enhancements

Enhanced User & Account Management

Implement features for managing unique account identifiers and comprehensive user profiles, including personal data and transaction history.

Secure Authentication Mechanisms

Introduce robust authentication protocols, such as hashed password storage and multi-factor authentication, to safeguard user accounts.

Persistent Data Storage Solutions

Integrate file I/O operations or database connectivity to ensure data persistence across application sessions, preventing data loss upon termination.

Scalable Multi-Account Framework

Develop a scalable architecture to support and manage an arbitrary number of user accounts concurrently, facilitating diverse financial operations.